
Compte Rendu

Travaux Pratiques

Bases de Données & PL/SQL

Matière : Bases de Données Avancées

Enseignant : Prof. H. Hrimech

Année académique : 2025 – 2026

filière : ISIBD-S6

Réalisé par :

SOUFI Manal

Table des matières

Introduction générale	3
1 TD de Réversion — Sous-Requêtes SQL	3
1.1 Exercice 1 — Base ETUDIANTS	3
1.2 Exercice 2	5
2 TP — Vues	7
2.1 Exercice 1 — Vues	7
3 TP — Les Vues en Bases de Données	9
3.1 Exercice 5 – Sécurité et abstraction	9
3.2 Exercice 6 – Analyse globale	9
4 TP — Variables, %TYPE, %ROWTYPE et Collections PL/SQL	10
4.1 Exercice Final de Synthèse	10
5 TP — PL/SQL : Structures de Contrôle et Boucles	11
5.1 Exercice 1 – Insertion de 1 à 100 dans une table	11
5.2 Exercice 2 – Somme de 1 à 1000	11
5.3 Exercice 3 – Nombres pairs entre 1 et 50	12
5.4 Exercice 4 – Factorielle d’un nombre	12
5.5 Exercice 5 – Insertion de 10 étudiants	12
5.6 Exercice 6 – Afficher de 10 à 1 avec WHILE	13
5.7 Exercice 7 – Augmenter le prix des produits de 10%	13
5.8 Exercice 8 – Supprimer les produits dont le prix < 50	14
5.9 Exercice 9 – Tester si un nombre est positif, négatif ou nul	14
5.10 Exercice 10 – Table des carrés de 1 à 20	14
6 TP — PL/SQL : Curseurs	14
6.1 Exercice 1 — Base Usine	15
6.2 Exercice 2 — Boucle FOR	16
7 TP — Curseurs PL/SQL Avancés	18
7.1 Exercice 1 – Structures conditionnelles IF-THEN-ELSE	18
7.2 Exercice 2 – Curseurs et attributs SQL%	20
8 TP — Procédures Stockées et Fonctions PL/SQL	22
8.1 Exercice 1 – Premier contact	22
8.2 Exercice 2 – Maximum de deux nombres	23
8.3 Exercice 3 – Permutation de deux nombres	23
8.4 Exercice 4 – Augmentation de salaire	25
8.5 Exercice 5 – Conversion dirhams en euros	26
8.6 Exercice 6 – Affichage des salaires supérieurs à un montant	27
8.7 Exercice 7 – Salaire moyen par département	28
8.8 Exercice 8 – Mise à jour d’adresse par nom	29
8.9 Exercice 9 – Nombre d’employés par département	30
8.10 Exercice 10 – Suppression des petits salaires	31
8.11 Tableau de synthèse	32

9 TP — Les Triggers PL/SQL sous Oracle	32
9.1 Exercices supplémentaires	33
Conclusion générale	35

Introduction générale

Ce compte rendu regroupe les neuf travaux pratiques réalisés au cours du semestre en bases de données relationnelles et PL/SQL sous Oracle. La progression couvre les sous-requêtes SQL, les vues, les variables et types PL/SQL, les boucles, les curseurs, les procédures et fonctions stockées, puis les triggers. Chaque chapitre rappelle l'énoncé et présente la solution commentée.

TP 1 – TD de Réversion — Sous-Requêtes SQL

1.1 Exercice 1 — Base ETUDIANTS

Énoncé

Schéma relationnel : ETUDIANT (Numetu, Nometu, Dtnaiss, Cdsexe)
SEXE (Cdsexe, Lbsexe)
ENSEIGNANT (Numens, Nomens, Grade, Ancien)
MATIERE (Numat, Nomat, Coeff, Numens)
NOTES (Numetu, Numat, Note)

Question 1 – Numéro, nom et sexe des étudiants sans date de naissance

```
1 SELECT e.Numetu, e.Nometu, s.Lbsexe
2 FROM ETUDIANT e
3 JOIN SEXE s ON e.Cdsexe = s.Cdsexe
4 WHERE e.Dtnaiss IS NULL;
```

Question 2 – Nom et date de naissance des étudiants dont la 2^e lettre est 'a'

```
1 SELECT Nometu, Dtnaiss
2 FROM ETUDIANT
3 WHERE Nometu LIKE '_a%';
```

Question 3 – Nom des matières avec coefficient entre 1 et 2

```
1 SELECT Nomat
2 FROM MATIERE
3 WHERE Coeff BETWEEN 1 AND 2;
```

Question 4 – Notes de l'étudiant 1 égales aux notes de l'étudiant 2

```

1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4     AND Note IN (SELECT Note FROM NOTES WHERE Numetu = 2);

```

Question 5 – Notes de l'étudiant 1 supérieures à celles de l'étudiant 2

```

1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4     AND Note > ANY (SELECT Note FROM NOTES WHERE Numetu = 2);

```

Question 6 – Notes de l'étudiant 1 inférieures à toutes celles de l'étudiant 9

```

1 SELECT Note
2 FROM NOTES
3 WHERE Numetu = 1
4     AND Note < ALL (SELECT Note FROM NOTES WHERE Numetu = 9);

```

Question 7 – Étudiants sans aucune note

```

1 SELECT *
2 FROM ETUDIANT
3 WHERE Numetu NOT IN (SELECT DISTINCT Numetu FROM NOTES);

```

Question 8 – Âge moyen des garçons et des filles au 1^{er} janvier 2000

```

1 SELECT s.Lbsexe,
2         AVG (TRUNC (MONTHS_BETWEEN (TO_DATE ('01/01/2000', 'DD/MM/YYYY'),
3                                     e.Dtnaiss) / 12)) AS age_moyen
4 FROM ETUDIANT e
5 JOIN SEXE s ON e.Cdsexe = s.Cdsexe
6 WHERE e.Dtnaiss IS NOT NULL
7 GROUP BY s.Lbsexe;

```

Question 9 – Nom et grade des enseignants d'histoire

```

1 SELECT e.Nomens, e.Grade
2 FROM ENSEIGNANT e
3 JOIN MATIERE m ON e.Numens = m.Numens
4 WHERE m.Nomat = 'Histoire';

```

Question 10 – Étudiants sans note en Sociologie

```
1 SELECT e.Numetu, e.Nometu
2 FROM ETUDIANT e
3 WHERE e.Numetu NOT IN (
4     SELECT n.Numetu
5     FROM NOTES n
6     JOIN MATIERE m ON n.Numat = m.Numat
7     WHERE m.Nomat = 'Sociologie'
8 );
```

1.2 Exercice 2

Énoncé

Schéma relationnel :

EMP(Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept)

DEPT(NumDept, NomDept, Lieu)

PROJET(CodeP, NomP)

PARTICIPATION(Matr, CodeP, Fonction)

Question 11 – Lieux des départements où des employés touchent une commission

```
1 SELECT DISTINCT d.Lieu
2 FROM DEPT d
3 WHERE d.NumDept IN (
4     SELECT NumDept FROM EMP WHERE Commission IS NOT NULL
5 );
```

Question 12 – Départements avec au moins un ingénieur

```
1 -- Solution par sous-requete
2 SELECT d.NomDept, d.Lieu
3 FROM DEPT d
4 WHERE d.NumDept IN (
5     SELECT NumDept FROM EMP WHERE Poste = 'Ingenieur'
6 );
7
8 -- Solution par jointure
9 SELECT DISTINCT d.NomDept, d.Lieu
10 FROM DEPT d
11 JOIN EMP e ON d.NumDept = e.NumDept
12 WHERE e.Poste = 'Ingenieur';
```

Question 13 – Départements sans ingénieur

```
1 SELECT d.NomDept, d.Lieu
2 FROM DEPT d
3 WHERE d.NumDept NOT IN (
4     SELECT NumDept FROM EMP WHERE Poste = 'Ingenieur'
5 );
```

Question 14 – Employés gagnant moins de 50% du salaire de leur supérieur

```
1 SELECT e.NomE, e.Salaire
2 FROM EMP e
3 WHERE e.Salaire < 0.5 * (
4     SELECT s.Salaire
5     FROM EMP s
6     WHERE s.Matr = e.Sup
7 );
```

Question 15 – Employés gagnant plus que tous les commerciaux

```
1 SELECT NomE, Salaire
2 FROM EMP
3 WHERE Salaire > ALL (
4     SELECT Salaire FROM EMP WHERE Poste = 'Commercial'
5 );
```

Question 16 – Employés gagnant plus que tous les commerciaux de leur département

```
1 -- Version incluant les departements sans commercial
2 SELECT e.NomE, e.Salaire, e.NumDept
3 FROM EMP e
4 WHERE e.Salaire > ALL (
5     SELECT c.Salaire
6     FROM EMP c
7     WHERE c.Poste = 'Commercial'
8     AND c.NumDept = e.NumDept
9 );
10
11 -- Version excluant les departements sans commercial
12 SELECT e.NomE, e.Salaire, e.NumDept
13 FROM EMP e
14 WHERE EXISTS (
15     SELECT 1 FROM EMP c
16     WHERE c.Poste = 'Commercial' AND c.NumDept = e.NumDept
17 )
```

```

18 AND e.Salaire > ALL (
19     SELECT c.Salaire
20     FROM EMP c
21     WHERE c.Poste = 'Commercial' AND c.NumDept = e.NumDept
22 );

```

Question 17 – Employés ayant le même poste et supérieur que BERGER

```

1 SELECT NomE, Poste, Sup
2 FROM EMP
3 WHERE (Poste, Sup) IN (
4     SELECT Poste, Sup FROM EMP WHERE NomE = 'BERGER'
5 )
6 AND NomE <> 'BERGER';

```

TP 2 – TP — Vues

Énoncé

Schéma relationnel :

EMP (Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, NumDept)

DEPT (NumDept, NomDept, Lieu)

PROJET (CodeP, NomP)

PARTICIPATION (Matr, CodeP, Fonction)

2.1 Exercice 1 — Vues

Question 1 – Créer la vue V_EMP

```

1 CREATE OR REPLACE VIEW V_EMP AS
2 SELECT e.Matr,
3        e.NomE,
4        e.NumDept,
5        NVL(e.Salaire, 0) + NVL(e.Commission, 0) AS GAINS,
6        d.Lieu
7 FROM EMP e
8 JOIN DEPT d ON e.NumDept = d.NumDept;

```

Question 2 – Sélectionner les lignes de V_EMP avec GAINS > 10 000

```

1 SELECT *
2 FROM V_EMP
3 WHERE GAINS > 10000;

```


Question 3 – Mise à jour du nom de l'employé MARTIN via la vue

```
1 UPDATE V_EMP
2 SET NomE = 'MARTIN_MODIF'
3 WHERE NomE = 'MARTIN';
4 COMMIT;
```

Question 4 – Vue VEMP10 sans option CHECK

```
1 -- Creation de la vue sans CHECK OPTION
2 CREATE OR REPLACE VIEW VEMP10 AS
3 SELECT * FROM EMP WHERE NumDept = 10;
4
5 -- Insertion d'un employe du departement 20
6 INSERT INTO VEMP10 (Matr, NomE, Poste, DatEmb, NumDept)
7 VALUES (9999, 'SOULIER', 'Employe', SYSDATE, 20);
8 COMMIT;
```

Question 5 – Recréer VEMP10 avec l'option CHECK

```
1 DROP VIEW VEMP10;
2
3 CREATE OR REPLACE VIEW VEMP10 AS
4 SELECT * FROM EMP
5 WHERE NumDept = 10
6 WITH CHECK OPTION CONSTRAINT vemp10_check;
```

Question 6 – Insertion de BALARD (département 30) et modification

```
1 -- Tentative d'insertion : ERREUR attendue
2 INSERT INTO VEMP10 (Matr, NomE, Poste, DatEmb, NumDept)
3 VALUES (8888, 'BALARD', 'Technicien', SYSDATE, 30);
4 -- ORA-01402 : violation de la clause WITH CHECK OPTION
5
6 -- Tentative de modifier le departement d'un employe visible
7 UPDATE VEMP10 SET NumDept = 20 WHERE Matr = 7782;
8 -- ORA-01402 : violation de la clause WITH CHECK OPTION
```

Question 7 – Salaires avec pourcentage par rapport au total du département

```
1 -- Solution avec une vue intermediaire
2 CREATE OR REPLACE VIEW V_TOTAL_DEPT AS
3 SELECT NumDept, SUM(Salaire) AS Total_Sal
```

```

4 FROM EMP
5 GROUP BY NumDept;
6
7 SELECT e.NomE,
8        e.NumDept,
9        e.Salaire,
10       ROUND(e.Salaire * 100 / t.Total_Sal, 2) AS Pourcentage
11 FROM EMP e
12 JOIN V_TOTAL_DEPT t ON e.NumDept = t.NumDept;
13
14 -- Solution sans vue (SELECT imbriquée dans le FROM)
15 SELECT e.NomE,
16        e.NumDept,
17        e.Salaire,
18       ROUND(e.Salaire * 100 / t.Total_Sal, 2) AS Pourcentage
19 FROM EMP e
20 JOIN (SELECT NumDept, SUM(Salaire) AS Total_Sal
21       FROM EMP
22       GROUP BY NumDept) t ON e.NumDept = t.NumDept;

```

TP 3 – TP — Les Vues en Bases de Données

Énoncé

Schéma : Departement(dept_id, nom_dept, ville) et
 Employe(emp_id, nom, salaire, dept_id)

3.1 Exercice 5 – Sécurité et abstraction

Vue publique sans salaire

```

1 CREATE VIEW emp_public AS
2 SELECT emp_id, nom, dept_id
3 FROM Employe;

```

3.2 Exercice 6 – Analyse globale

Réponse

1. Architecture basée sur les vues pour la sécurité :

- Vue v_manager_dept : filtrée par département, protégée par WITH CHECK OPTION.
- Vue v_rh_complet : accès complet pour le service RH (GRANT SELECT ON v_rh_complet TO rh_role).
- Vue v_employe_public : sans colonne salaire pour les employés ordinaires.

2. Limites des vues classiques : Les vues classiques sont réévaluées à chaque

requête. Sur des tables de plusieurs centaines de milliers de lignes, les jointures et agrégations induisent des temps de réponse élevés.

3. Vues matérialisées : Une vue matérialisée stocke physiquement le résultat et peut être rafraîchie périodiquement (ON COMMIT ou ON DEMAND). Elle améliore considérablement les performances en lecture mais engendre une surcharge lors des mises à jour.

4. Vues vs GRANT/REVOKE : Les vues offrent une abstraction fine (cacher des colonnes, filtrer des lignes). GRANT/REVOKE gère les droits sur les objets entiers. Les deux sont complémentaires.

5. Les vues seules ne suffisent pas : Une sécurité complète requiert aussi la gestion des privilèges, le chiffrement des données sensibles, l'audit et le contrôle des connexions.

6. Stratégie d'optimisation :

- Index sur les colonnes de jointure et de filtrage.
- Partitionnement des grandes tables.
- Vues matérialisées pour les agrégats fréquemment consultés.
- Statistiques à jour pour l'optimiseur (DBMS_STATS).

TP 4 – TP — Variables, %TYPE, %ROWTYPE et Collections PL/SQL

4.1 Exercice Final de Synthèse

Travail demandé :

- Créer un RECORD personnalisé.
- Créer une TABLE de ce RECORD.
- Insérer 5 employés.
- Calculer le salaire moyen.
- Identifier le salaire maximum.

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     TYPE emp_rec IS RECORD (
5         id      NUMBER,
6         nom     VARCHAR2(50),
7         salaire NUMBER
8     );
9
10    TYPE emp_table IS TABLE OF emp_rec
11    INDEX BY BINARY_INTEGER;
12
13    v_emps    emp_table;
14    total     NUMBER := 0;
15    moyenne   NUMBER;
16    max_sal   NUMBER := 0;
```

```

17 BEGIN
18     v_emps(1) := emp_rec(1, 'Ali',      3000);
19     v_emps(2) := emp_rec(2, 'Sara',     4500);
20     v_emps(3) := emp_rec(3, 'Lina',     5000);
21     v_emps(4) := emp_rec(4, 'Adam',     3500);
22     v_emps(5) := emp_rec(5, 'Youssef',  6000);
23
24     FOR i IN 1..5 LOOP
25         total := total + v_emps(i).salaire;
26         IF v_emps(i).salaire > max_sal THEN
27             max_sal := v_emps(i).salaire;
28         END IF;
29     END LOOP;
30
31     moyenne := total / 5;
32
33     DBMS_OUTPUT.PUT_LINE('Salaire moyen   : ' || moyenne);
34     DBMS_OUTPUT.PUT_LINE('Salaire maximum : ' || max_sal);
35 END;
36 /

```

TP 5 – TP — PL/SQL : Structures de Contrôle et Boucles

5.1 Exercice 1 – Insertion de 1 à 100 dans une table

```

1 CREATE TABLE Nombre (n NUMBER);
2
3 DECLARE
4     i NUMBER := 1;
5 BEGIN
6     LOOP
7         INSERT INTO Nombre VALUES (i);
8         i := i + 1;
9         EXIT WHEN i > 100;
10    END LOOP;
11    COMMIT;
12    DBMS_OUTPUT.PUT_LINE('100 lignes inserees. ');
13 END;
14 /

```

5.2 Exercice 2 – Somme de 1 à 1000

```

1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     total NUMBER := 0;
5 BEGIN

```

```

6      FOR i IN 1..1000 LOOP
7          total := total + i;
8      END LOOP;
9      DBMS_OUTPUT.PUT_LINE('Somme 1 a 1000 = ' || total);
10 END;
11 /
12 -- Resultat attendu : 500500

```

5.3 Exercice 3 – Nombres pairs entre 1 et 50

```

1 SET SERVEROUTPUT ON;
2
3 BEGIN
4     FOR i IN 1..50 LOOP
5         IF MOD(i, 2) = 0 THEN
6             DBMS_OUTPUT.PUT_LINE(i);
7         END IF;
8     END LOOP;
9 END;
10 /

```

5.4 Exercice 4 – Factorielle d'un nombre

```

1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     n          NUMBER := 7;    -- modifier selon besoin
5     facteur    NUMBER := 1;
6 BEGIN
7     IF n < 0 THEN
8         DBMS_OUTPUT.PUT_LINE('Factorielle non definie pour n < 0');
9     ELSIF n = 0 THEN
10        DBMS_OUTPUT.PUT_LINE('0! = 1');
11    ELSE
12        FOR i IN 1..n LOOP
13            facteur := facteur * i;
14        END LOOP;
15        DBMS_OUTPUT.PUT_LINE(n || '! = ' || facteur);
16    END IF;
17 END;
18 /

```

5.5 Exercice 5 – Insertion de 10 étudiants

```

1 CREATE TABLE Etudiant (
2     id      NUMBER PRIMARY KEY,

```

```

3      nom  VARCHAR2(50),
4      note NUMBER(4,2)
5  );
6
7  DECLARE
8      noms SYS.ODCIVARCHAR2LIST := SYS.ODCIVARCHAR2LIST(
9          'Ahmed', 'Sara', 'Karim', 'Nadia', 'Youssef',
10         'Fatima', 'Omar', 'Leila', 'Hassan', 'Amina'
11     );
12 BEGIN
13     FOR i IN 1..10 LOOP
14         INSERT INTO Etudiant VALUES (i, noms(i), 10 + i);
15     END LOOP;
16     COMMIT;
17     DBMS_OUTPUT.PUT_LINE('10 etudiants inseres avec succes.');

```

5.6 Exercice 6 – Afficher de 10 à 1 avec WHILE

```

1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     i NUMBER := 10;
5 BEGIN
6     WHILE i >= 1 LOOP
7         DBMS_OUTPUT.PUT_LINE(i);
8         i := i - 1;
9     END LOOP;
10 END;
11 /

```

5.7 Exercice 7 – Augmenter le prix des produits de 10%

```

1 CREATE TABLE Produit (
2     id    NUMBER PRIMARY KEY,
3     nom   VARCHAR2(50),
4     prix  NUMBER(8,2)
5 );
6
7 BEGIN
8     UPDATE Produit SET prix = prix * 1.10;
9     COMMIT;
10    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' produits mis a jour.');

```

5.8 Exercice 8 – Supprimer les produits dont le prix < 50

```
1 BEGIN
2     DELETE FROM Produit WHERE prix < 50;
3     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' produit(s) supprime(s).');
4     COMMIT;
5 END;
6 /
```

5.9 Exercice 9 – Tester si un nombre est positif, négatif ou nul

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     n NUMBER := -7;    -- modifier selon besoin
5 BEGIN
6     IF n > 0 THEN
7         DBMS_OUTPUT.PUT_LINE(n || ' est positif.');
```

```
8     ELSIF n < 0 THEN
9         DBMS_OUTPUT.PUT_LINE(n || ' est negatif.');
```

```
10    ELSE
11        DBMS_OUTPUT.PUT_LINE('Le nombre est nul.');
```

```
12    END IF;
13 END;
14 /
```

5.10 Exercice 10 – Table des carrés de 1 à 20

```
1 CREATE TABLE Carre (nombre NUMBER, carre NUMBER);
2
3 BEGIN
4     FOR i IN 1..20 LOOP
5         INSERT INTO Carre VALUES (i, i * i);
6     END LOOP;
7     COMMIT;
8     DBMS_OUTPUT.PUT_LINE('Table des carres inseree.');
```

```
9 END;
10 /
```

TP 6 – TP — PL/SQL : Curseurs

6.1 Exercice 1 — Base Usine

Énoncé

Commande(Numcmd, Datcmd, #Numfour, Mncmd)
Fournisseurs(Numfour, Adrfour)

Question 1 – Liste de toutes les commandes

```
1 DECLARE
2     CURSOR c1 IS SELECT * FROM commande;
3 BEGIN
4     FOR i IN c1 LOOP
5         DBMS_OUTPUT.PUT_LINE(
6             i.numcmd || ' ' || i.datcmd || ' '
7             || i.mncmd || ' ' || i.numfour
8         );
9     END LOOP;
10 END;
11 /
```

Question 2 – Commandes dont le montant dépasse la moyenne

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     CURSOR c1 IS
5         SELECT * FROM commande
6         WHERE mncmd > (SELECT AVG(mncmd) FROM commande);
7 BEGIN
8     FOR i IN c1 LOOP
9         DBMS_OUTPUT.PUT_LINE(
10            i.numcmd || ' ' || i.datcmd || ' '
11            || i.mncmd || ' ' || i.numfour
12        );
13    END LOOP;
14 END;
15 /
```

Question 3 – Commandes avec fournisseur parisien

```
1 DECLARE
2     CURSOR c1 IS
3         SELECT c.numcmd, c.datcmd, c.numfour, c.mncmd
4         FROM   commande c
5         JOIN   fournisseurs f ON c.numfour = f.numfour
6         WHERE  f.adrfour = 'Paris';
```



```

7 BEGIN
8     FOR i IN c1 LOOP
9         DBMS_OUTPUT.PUT_LINE(
10             i.numcmd || ' ' || i.datcmd || ' '
11             || i.mncmd || ' ' || i.numfour
12         );
13     END LOOP;
14 END;
15 /

```

6.2 Exercice 2 — Boucle FOR

Énoncé

Emp (NumE, NomE, Fonction, NumS, Embauche, Salaire, Comm, NumD)
 Dept (NumD, NomD, Lieu)

Question 1 – Employés avec commission, tri décroissant

```

1 DECLARE
2     CURSOR c1 IS
3         SELECT NomE, Comm
4         FROM Emp
5         WHERE Comm IS NOT NULL
6         ORDER BY Comm DESC;
7 BEGIN
8     FOR i IN c1 LOOP
9         DBMS_OUTPUT.PUT_LINE(i.NomE || ' ' || i.Comm);
10    END LOOP;
11 END;
12 /

```

Question 2 – Personnes embauchées depuis le 01/09/2006

```

1 DECLARE
2     CURSOR c2 IS
3         SELECT NomE
4         FROM Emp
5         WHERE Embauche >= TO_DATE('01/09/2006', 'DD/MM/YYYY');
6 BEGIN
7     FOR i IN c2 LOOP
8         DBMS_OUTPUT.PUT_LINE(i.NomE);
9     END LOOP;
10 END;
11 /

```

Question 3 – Employés travaillant à Créteil

```
1 DECLARE
2     CURSOR c3 IS
3         SELECT e.NomE
4         FROM Emp e
5         JOIN Dept d ON e.NumD = d.NumD
6         WHERE d.Lieu = 'Creteil';
7 BEGIN
8     FOR i IN c3 LOOP
9         DBMS_OUTPUT.PUT_LINE(i.NomE);
10    END LOOP;
11 END;
12 /
```

Question 4 – Subordonnés de Guimezanes

```
1 DECLARE
2     CURSOR c4 IS
3         SELECT e2.NomE
4         FROM Emp e1
5         JOIN Emp e2 ON e2.NumS = e1.NumE
6         WHERE e1.NomE = 'Guimezanes';
7 BEGIN
8     FOR i IN c4 LOOP
9         DBMS_OUTPUT.PUT_LINE(i.NomE);
10    END LOOP;
11 END;
12 /
```

Question 5 – Moyenne des salaires

```
1 DECLARE
2     CURSOR c5 IS SELECT AVG(salaire) AS moy FROM Emp;
3 BEGIN
4     FOR i IN c5 LOOP
5         DBMS_OUTPUT.PUT_LINE('Salaire moyen = ' || ROUND(i.moy, 2));
6     END LOOP;
7 END;
8 /
```

Question 6 – Nombre de commissions non nulles

```
1 DECLARE
2     CURSOR c6 IS SELECT COUNT(Comm) AS nb FROM Emp;
3 BEGIN
4     FOR i IN c6 LOOP
```

```

5         DBMS_OUTPUT.PUT_LINE('Nombre de commissions = ' || i.nb);
6     END LOOP;
7 END;
8 /

```

Question 7 – Employés gagnant plus que la moyenne

```

1 DECLARE
2     CURSOR c7 IS
3         SELECT NomE, Salaire
4         FROM Emp
5         WHERE Salaire > (SELECT AVG(Salaire) FROM Emp);
6 BEGIN
7     FOR i IN c7 LOOP
8         DBMS_OUTPUT.PUT_LINE(i.NomE || ' Salaire = ' || i.Salaire);
9     END LOOP;
10 END;
11 /

```

TP 7 – TP — Curseurs PL/SQL Avancés

7.1 Exercice 1 – Structures conditionnelles IF-THEN-ELSE

Réorganiser deux variables (petit/grand)

```

1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     small_nbr NUMBER := 25;
5     big_nbr   NUMBER := 5;
6     tmp       NUMBER;
7 BEGIN
8     IF small_nbr > big_nbr THEN
9         tmp := small_nbr;
10        small_nbr := big_nbr;
11        big_nbr := tmp;
12    END IF;
13
14    DBMS_OUTPUT.PUT_LINE('small_nbr = ' || small_nbr);
15    DBMS_OUTPUT.PUT_LINE('big_nbr   = ' || big_nbr);
16 END;
17 /

```

Vérifier si un nombre est pair ou impair

```

1 DECLARE
2     nbr NUMBER := 3;
3 BEGIN

```

```

4      IF MOD(nbr, 2) = 0 THEN
5          DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est pair.');
```

```

6      ELSE
7          DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est impair.');
```

```

8      END IF;
9  END;
10 /
```

Vérifier si une date tombe le week-end

```

1  DECLARE
2      date1 DATE := TO_DATE('02-Mar-2024', 'DD-Mon-YYYY');
3      jour VARCHAR2(20);
4  BEGIN
5      jour := TO_CHAR(date1, 'DAY', 'NLS_DATE_LANGUAGE=ENGLISH');
```

```

6
7      IF jour IN ('SATURDAY', 'SUNDAY') THEN
8          DBMS_OUTPUT.PUT_LINE(
9              'Le ' || TO_CHAR(date1, 'DD/MM/YYYY')
10             || ' (' || TRIM(jour) || ') tombe le week-end.'
11         );
12      ELSE
13          DBMS_OUTPUT.PUT_LINE(
14              'Le ' || TO_CHAR(date1, 'DD/MM/YYYY')
15             || ' (' || TRIM(jour) || ') est un jour ouvre.'
16         );
17      END IF;
18  END;
19 /
```

Copier les enregistrements de **clients** vers **tmp_clients**

```

1  -- Creation de la table de destination
2  CREATE TABLE tmp_clients AS SELECT * FROM clients WHERE 1=0;
3
4  DECLARE
5      CURSOR c_clients IS SELECT * FROM clients;
6  BEGIN
7      FOR rec IN c_clients LOOP
8          INSERT INTO tmp_clients VALUES rec;
9      END LOOP;
10     COMMIT;
11     DBMS_OUTPUT.PUT_LINE('Copie terminee : '
12                          || SQL%ROWCOUNT || ' lignes.');
```

```

13  END;
14  /
```

7.2 Exercice 2 – Curseurs et attributs SQL%

Afficher tous les noms de pays

```
1 DECLARE
2     CURSOR c_pays IS SELECT nom FROM pays;
3 BEGIN
4     FOR rec IN c_pays LOOP
5         DBMS_OUTPUT.PUT_LINE(rec.nom);
6     END LOOP;
7 END;
8 /
```

Afficher identifiants et noms des pays

```
1 DECLARE
2     CURSOR c_pays IS SELECT pays_id, nom FROM pays;
3 BEGIN
4     DBMS_OUTPUT.PUT_LINE(' ID      | Nom du pays');
5     DBMS_OUTPUT.PUT_LINE('-----+-----');
6     FOR rec IN c_pays LOOP
7         DBMS_OUTPUT.PUT_LINE(rec.pays_id || ' | ' || rec.nom);
8     END LOOP;
9 END;
10 /
```

SQL%ROWCOUNT après DELETE

```
1 BEGIN
2     DELETE FROM pays WHERE pays_id = 1001;
3
4     IF SQL%FOUND THEN
5         DBMS_OUTPUT.PUT_LINE(
6             SQL%ROWCOUNT || ' ligne(s) supprimee(s).';
7         );
8     ELSE
9         DBMS_OUTPUT.PUT_LINE('Aucune ligne supprimee.');
```

SQL%NOTFOUND après UPDATE

```
1 BEGIN
2     UPDATE pays SET nom = 'Algerie' WHERE pays_id = 9999;
3
```

```

4      IF SQL%NOTFOUND THEN
5          DBMS_OUTPUT.PUT_LINE('Aucune ligne modifiee (pays
              inexistant).');
6      ELSE
7          DBMS_OUTPUT.PUT_LINE('Mise a jour effectuee.');
```

Afficher employés avec leur département

```

1  DECLARE
2      CURSOR c_emp IS
3          SELECT e.id, e.nom, e.prenom, d.nom AS dept
4          FROM employees e
5          JOIN departements d ON e.departement_id = d.dep_id;
6  BEGIN
7      FOR rec IN c_emp LOOP
8          DBMS_OUTPUT.PUT_LINE(
9              rec.id || ' | ' || rec.nom || ' ' || rec.prenom
10             || ' | ' || rec.dept
11         );
12     END LOOP;
13 END;
14 /
```

Augmentation de salaire avec WHERE CURRENT OF

```

1  DECLARE
2      CURSOR c_emp IS
3          SELECT id, salaire
4          FROM employees
5          WHERE departement_id = 8
6          FOR UPDATE OF salaire;
7  BEGIN
8      FOR rec IN c_emp LOOP
9          IF rec.salaire < 5000 THEN
10             UPDATE employees
11             SET salaire = salaire + 100
12             WHERE CURRENT OF c_emp;
13         ELSE
14             UPDATE employees
15             SET salaire = salaire + 50
16             WHERE CURRENT OF c_emp;
17         END IF;
18     END LOOP;
19     COMMIT;
20     DBMS_OUTPUT.PUT_LINE('Salaires mis a jour pour le dept 8.');
```

```
21 END;  
22 /
```

TP 8 – TP — Procédures Stockées et Fonctions PL/SQL

Énoncé

Table utilisée : EMPLOYES (ENUM, ENAME, SALARY, ADDRESS, DEPT)

8.1 Exercice 1 – Premier contact

1. Bloc anonyme affichant Hello

```
1 BEGIN  
2     DBMS_OUTPUT.PUT_LINE('Hello');  
3 END;  
4 /
```

2. Fonction retournant le carré d'un nombre

```
1 CREATE OR REPLACE FUNCTION carre (n IN NUMBER)  
2 RETURN NUMBER IS  
3 BEGIN  
4     RETURN n * n;  
5 END;  
6 /  
7  
8 -- Test  
9 SELECT carre(5) FROM DUAL;  
10 -- Resultat : 25
```

Questions supplémentaires

1. Que se passe-t-il si on omet RETURN dans une fonction ?

Si on omet RETURN dans le corps d'une fonction, Oracle génère une erreur de compilation PLS-00503. La fonction ne peut pas être créée car le compilateur exige obligatoirement une instruction RETURN dans toute fonction PL/SQL.

2. Une procédure peut-elle retourner une valeur via un paramètre OUT ?

Oui. Une procédure peut retourner une ou plusieurs valeurs via des paramètres OUT ou IN OUT. La différence avec une fonction est qu'une procédure ne peut pas être appelée directement dans une expression SQL (SELECT, WHERE), contrairement à une fonction.

3. Pourquoi utilise-t-on FROM DUAL pour tester une fonction ?

DUAL est une table virtuelle à une seule ligne fournie par Oracle. Comme SELECT exige une clause FROM, on utilise DUAL lorsqu'on veut tester une fonction sans interroger une vraie table.

8.2 Exercice 2 – Maximum de deux nombres

```
1 CREATE OR REPLACE PROCEDURE max_deux (
2     a    IN  NUMBER,
3     b    IN  NUMBER,
4     res  OUT NUMBER
5 ) IS
6 BEGIN
7     IF a >= b THEN res := a;
8     ELSE          res := b;
9     END IF;
10 END;
11 /
12
13 -- Test
14 DECLARE
15     resultat NUMBER;
16 BEGIN
17     max_deux(17, 42, resultat);
18     DBMS_OUTPUT.PUT_LINE('Maximum = ' || resultat);
19 END;
20 /
```

Questions supplémentaires

1. Que se passe-t-il si on n'initialise pas le paramètre OUT ?

Si aucun chemin d'exécution n'affecte le paramètre OUT, sa valeur reste NULL pour l'appelant. Il n'y a pas d'erreur à la compilation mais le résultat sera incorrect.

2. Version fonction du calcul du maximum

```
1 CREATE OR REPLACE FUNCTION fn_max_deux (
2     a IN NUMBER,
3     b IN NUMBER
4 ) RETURN NUMBER IS
5 BEGIN
6     IF a >= b THEN RETURN a;
7     ELSE          RETURN b;
8     END IF;
9 END;
10 /
11
12 SELECT fn_max_deux(17, 42) FROM DUAL;
```

3. Quelle est la portée des paramètres OUT ?

Un paramètre OUT est local au sous-programme pendant son exécution. Sa valeur est transmise à la variable de l'appelant uniquement au retour normal de la procédure. En cas d'exception non gérée, la valeur n'est pas transmise.

8.3 Exercice 3 – Permutation de deux nombres


```

1 CREATE OR REPLACE PROCEDURE permuter (
2     a IN OUT NUMBER,
3     b IN OUT NUMBER
4 ) IS
5     tmp NUMBER;
6 BEGIN
7     tmp := a;
8     a   := b;
9     b   := tmp;
10 END;
11 /
12
13 -- Test
14 DECLARE
15     x NUMBER := 10;
16     y NUMBER := 20;
17 BEGIN
18     permuter(x, y);
19     DBMS_OUTPUT.PUT_LINE('x = ' || x || ', y = ' || y);
20     -- Resultat : x = 20, y = 10
21 END;
22 /

```

Questions supplémentaires

1. Que se passerait-il si on utilisait IN pour les deux paramètres ?

Avec IN, les paramètres sont en lecture seule. Oracle générerait une erreur de compilation PLS-00363 : on ne peut pas affecter une valeur à un paramètre IN.

2. Autre exemple concret de l'utilité de IN OUT

Une procédure qui incrémente un compteur :

```

1 CREATE OR REPLACE PROCEDURE incrementer (
2     p_compteur IN OUT NUMBER,
3     p_pas      IN      NUMBER DEFAULT 1
4 ) IS
5 BEGIN
6     p_compteur := p_compteur + p_pas;
7 END;
8 /

```

3. Peut-on permuter deux colonnes d'une table avec une procédure ?

Oui, en utilisant une variable temporaire :

```

1 CREATE OR REPLACE PROCEDURE permuter_colonnes (
2     p_id IN NUMBER
3 ) IS
4     tmp NUMBER;
5 BEGIN
6     SELECT col1 INTO tmp FROM ma_table WHERE id = p_id;
7     UPDATE ma_table SET col1 = col2, col2 = tmp
8     WHERE id = p_id;

```

```

9      COMMIT;
10 END;
11 /

```

8.4 Exercice 4 – Augmentation de salaire

```

1 CREATE OR REPLACE PROCEDURE augmenter_salaire (
2     p_nom   IN EMPLOYES.ENAME%TYPE,
3     p_taux  IN NUMBER
4 ) IS
5     v_count NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_count
8     FROM EMPLOYES WHERE ENAME = p_nom;
9
10    IF v_count = 0 THEN
11        DBMS_OUTPUT.PUT_LINE('Employe ' || p_nom || ' introuvable.');
```

```

12    ELSIF v_count > 1 THEN
13        DBMS_OUTPUT.PUT_LINE('Plusieurs employes portent ce nom.');
```

```

14    ELSE
15        UPDATE EMPLOYES
16        SET SALARY = SALARY * (1 + p_taux / 100)
17        WHERE ENAME = p_nom;
18        COMMIT;
19        DBMS_OUTPUT.PUT_LINE('Salaire de ' || p_nom
20                                || ' augmente de ' || p_taux || '%.');
```

```

21    END IF;
22 END;
23 /

```

Questions supplémentaires

1. Pourquoi est-il prudent d'utiliser ROWNUM = 1 ou la clé primaire ?

Le nom n'est pas unique dans la table EMPLOYES (deux employés peuvent s'appeler Ahmed). Un UPDATE sur le nom modifierait toutes les lignes correspondantes. Utiliser la clé primaire ENUM garantit qu'une seule ligne est modifiée.

2. Que vaut SQL%ROWCOUNT après un UPDATE ?

SQL%ROWCOUNT contient le nombre de lignes affectées par le dernier ordre DML. Si l'UPDATE a touché 3 lignes, SQL%ROWCOUNT vaut 3. Si aucune ligne, il vaut 0.

3. Variante par clé primaire ENUM

```

1 CREATE OR REPLACE PROCEDURE augmenter_salaire_id (
2     p_enum  IN EMPLOYES.ENUM%TYPE,
3     p_taux  IN NUMBER
4 ) IS
5 BEGIN
6     UPDATE EMPLOYES
7     SET SALARY = SALARY * (1 + p_taux / 100)
8     WHERE ENUM = p_enum;

```

```

9
10     IF SQL%ROWCOUNT = 0 THEN
11         DBMS_OUTPUT.PUT_LINE('Aucun employe avec ENUM = ' || p_enum);
12     ELSE
13         COMMIT;
14         DBMS_OUTPUT.PUT_LINE('Salaire mis a jour.');
```

8.5 Exercice 5 – Conversion dirhams en euros

```

1 CREATE OR REPLACE FUNCTION dh_vers_euros (
2     p_montant IN NUMBER
3 ) RETURN NUMBER IS
4     c_taux CONSTANT NUMBER := 11;
5 BEGIN
6     RETURN ROUND(p_montant / c_taux, 2);
7 END;
8 /
9
10 SELECT dh_vers_euros(1100) FROM DUAL;
11 -- Resultat : 100
```

Questions supplémentaires

1. Une fonction peut-elle contenir un COMMIT ou ROLLBACK ?

Techniquement oui, mais c'est fortement déconseillé. Une fonction appelée dans un SELECT pourrait valider ou annuler une transaction en cours sans que l'appelant le sache, ce qui provoque des effets de bord imprévisibles. Oracle interdit même le COMMIT dans une fonction appelée depuis SQL.

2. Version avec taux variable

```

1 CREATE OR REPLACE FUNCTION dh_vers_euros_v2 (
2     p_montant IN NUMBER,
3     p_taux    IN NUMBER DEFAULT 11
4 ) RETURN NUMBER IS
5 BEGIN
6     RETURN ROUND(p_montant / p_taux, 2);
7 END;
8 /
9
10 -- Appel avec taux par défaut
11 SELECT dh_vers_euros_v2(550) FROM DUAL;
12 -- Appel avec taux personnalisée
13 SELECT dh_vers_euros_v2(550, 10.5) FROM DUAL;
```

3. Peut-on utiliser cette fonction dans une clause WHERE ?

Oui, car c'est une fonction pure (pas de DML, pas de transaction) :

```

1 SELECT ENAME, SALARY
2 FROM EMPLOYES
3 WHERE dh_vers_euros(SALARY) > 500;

```

8.6 Exercice 6 – Affichage des salaires supérieurs à un montant

```

1 CREATE OR REPLACE PROCEDURE employes_salaire_sup (
2     p_seuil IN NUMBER,
3     p_ordre IN VARCHAR2 DEFAULT 'ASC'
4 ) IS
5     CURSOR c_asc IS
6         SELECT ENAME, SALARY, DEPT FROM EMPLOYES
7         WHERE SALARY > p_seuil ORDER BY SALARY ASC;
8     CURSOR c_desc IS
9         SELECT ENAME, SALARY, DEPT FROM EMPLOYES
10        WHERE SALARY > p_seuil ORDER BY SALARY DESC;
11 BEGIN
12     IF p_seuil < 0 THEN
13         DBMS_OUTPUT.PUT_LINE('Seuil invalide (negatif).');
14         RETURN;
15     END IF;
16
17     IF UPPER(p_ordre) = 'DESC' THEN
18         FOR rec IN c_desc LOOP
19             DBMS_OUTPUT.PUT_LINE(
20                 rec.ENAME || ' | ' || rec.DEPT || ' | ' || rec.SALARY
21             );
22         END LOOP;
23     ELSE
24         FOR rec IN c_asc LOOP
25             DBMS_OUTPUT.PUT_LINE(
26                 rec.ENAME || ' | ' || rec.DEPT || ' | ' || rec.SALARY
27             );
28         END LOOP;
29     END IF;
30 END;
31 /

```

Questions supplémentaires

1. Comment faire la même chose sans curseur explicite ?

```

1 CREATE OR REPLACE PROCEDURE employes_salaire_sup_v2 (
2     p_seuil IN NUMBER
3 ) IS
4 BEGIN
5     FOR rec IN (SELECT ENAME, SALARY, DEPT FROM EMPLOYES
6                 WHERE SALARY > p_seuil ORDER BY SALARY DESC)
7     LOOP

```

```

8      DBMS_OUTPUT.PUT_LINE (
9          rec.ENAME || ' | ' || rec.DEPT || ' | ' || rec.SALARY
10     );
11     END LOOP;
12 END;
13 /

```

2. Paramètre de tri ajouté

Déjà implémenté ci-dessus avec le paramètre `p_ordre IN VARCHAR2 DEFAULT 'ASC'`.

3. Que se passe-t-il si le montant est négatif?

La procédure détecte le cas avec `IF p_seuil < 0` et affiche un message d'erreur avant de sortir avec `RETURN`, sans exécuter la requête.

8.7 Exercice 7 – Salaire moyen par département

```

1 CREATE OR REPLACE FUNCTION salaire_moyen_dept (
2     p_dept IN EMPLOYES.DEPT%TYPE
3 ) RETURN NUMBER IS
4     v_moy NUMBER;
5 BEGIN
6     SELECT AVG(SALARY) INTO v_moy
7     FROM EMPLOYES WHERE DEPT = p_dept;
8     RETURN NVL(v_moy, NULL);
9 EXCEPTION
10    WHEN NO_DATA_FOUND THEN RETURN NULL;
11 END;
12 /
13
14 SELECT salaire_moyen_dept('D1') FROM DUAL;

```

Questions supplémentaires

1. Pourquoi utiliser NVL ou gérer NO_DATA_FOUND ?

AVG sur un ensemble vide retourne NULL (pas d'exception). Cependant, si le département n'existe pas du tout, `SELECT AVG INTO` ne déclenche pas `NO_DATA_FOUND` car AVG retourne toujours une ligne (avec NULL). On gère quand même l'exception par sécurité et on utilise NVL pour une valeur par défaut propre.

2. Modifier la fonction pour retourner NULL si aucun employé

La version actuelle retourne déjà NULL via `NVL(v_moy, NULL)` si le département est vide.

3. Peut-on appeler cette fonction dans une procédure qui met à jour une table de statistiques ?

```

1 CREATE OR REPLACE PROCEDURE maj_stats_dept IS
2 BEGIN
3     FOR rec IN (SELECT DISTINCT DEPT FROM EMPLOYES) LOOP
4         UPDATE stats_dept
5         SET salaire_moyen = salaire_moyen_dept(rec.DEPT)

```

```

6         WHERE dept = rec.DEPT;
7     END LOOP;
8     COMMIT;
9 END;
10 /

```

8.8 Exercice 8 – Mise à jour d'adresse par nom

```

1 CREATE OR REPLACE PROCEDURE maj_adresse (
2     p_nom          IN EMPLOYES.ENAME%TYPE,
3     p_new_adresse  IN EMPLOYES.ADDRESS%TYPE
4 ) IS
5 BEGIN
6     IF TRIM(p_new_adresse) IS NULL THEN
7         RAISE_APPLICATION_ERROR(-20001,
8             'Adresse vide non autorisee. ');
9     END IF;
10
11    UPDATE EMPLOYES
12    SET ADDRESS = p_new_adresse
13    WHERE ENAME = p_nom AND ROWNUM = 1;
14
15    IF SQL%ROWCOUNT = 0 THEN
16        DBMS_OUTPUT.PUT_LINE('Employe ' || p_nom || ' non trouve. ');
17    ELSE
18        COMMIT;
19        DBMS_OUTPUT.PUT_LINE('Adresse mise a jour pour ' || p_nom);
20    END IF;
21 END;
22 /

```

Questions supplémentaires

1. Comment garantir qu'une seule ligne est mise à jour ?

On utilise `ROWNUM = 1` pour limiter à une seule ligne si plusieurs employés portent le même nom, ou mieux, on passe la clé primaire `ENUM` en paramètre.

2. Variante avec ENUM

```

1 CREATE OR REPLACE PROCEDURE maj_adresse_id (
2     p_enum          IN EMPLOYES.ENUM%TYPE,
3     p_new_adresse  IN EMPLOYES.ADDRESS%TYPE
4 ) IS
5 BEGIN
6     IF TRIM(p_new_adresse) IS NULL THEN
7         RAISE_APPLICATION_ERROR(-20001,
8             'Adresse vide non autorisee. ');
9     END IF;
10    UPDATE EMPLOYES
11    SET ADDRESS = p_new_adresse

```

```

12 WHERE ENUM = p_enum;
13 IF SQL%ROWCOUNT = 0 THEN
14     DBMS_OUTPUT.PUT_LINE('ENUM ' || p_enum || ' introuvable. ');
15 ELSE
16     COMMIT;
17 END IF;
18 END;
19 /

```

3. Vérification que l'adresse n'est pas vide

Déjà implémentée avec `IF TRIM(p_new_adresse) IS NULL` dans les deux versions.

8.9 Exercice 9 – Nombre d'employés par département

```

1 CREATE OR REPLACE FUNCTION nb_employes_dept (
2     p_dept IN EMPLOYES.DEPT%TYPE
3 ) RETURN NUMBER IS
4     v_count NUMBER;
5 BEGIN
6     SELECT COUNT(*) INTO v_count
7     FROM EMPLOYES WHERE DEPT = p_dept;
8     RETURN v_count;
9 END;
10 /
11
12 -- Afficher pour chaque departement
13 BEGIN
14     FOR rec IN (SELECT DISTINCT DEPT FROM EMPLOYES ORDER BY DEPT)
15     LOOP
16         DBMS_OUTPUT.PUT_LINE(
17             'Dept ' || rec.DEPT || ' : '
18             || nb_employes_dept(rec.DEPT) || ' employe(s)'
19         );
20     END LOOP;
21 END;
22 /

```

Questions supplémentaires

1. Différence entre COUNT(*) et COUNT(ENUM) ?

`COUNT(*)` compte toutes les lignes y compris celles avec des NULL. `COUNT(ENUM)` compte uniquement les lignes où `ENUM` n'est pas NULL. Pour une clé primaire (jamais NULL), les deux donnent le même résultat.

2. Retourner aussi le salaire moyen avec un type enregistrement

```

1 CREATE OR REPLACE PROCEDURE stats_dept (
2     p_dept      IN  EMPLOYES.DEPT%TYPE,
3     p_nb        OUT NUMBER,
4     p_sal_moy   OUT NUMBER

```

```

5 ) IS
6 BEGIN
7     SELECT COUNT(*), AVG(SALARY)
8     INTO p_nb, p_sal_moy
9     FROM EMPLOYES
10    WHERE DEPT = p_dept;
11 END;
12 /
13
14 -- Test
15 DECLARE
16     v_nb    NUMBER;
17     v_moy   NUMBER;
18 BEGIN
19     stats_dept('D1', v_nb, v_moy);
20     DBMS_OUTPUT.PUT_LINE('Nb : ' || v_nb
21                          || ' | Moy : ' || ROUND(v_moy, 2));
22 END;
23 /

```

3. Procédure affichant pour chaque département le nombre d'employés

```

1 CREATE OR REPLACE PROCEDURE afficher_stats_tous_depts IS
2 BEGIN
3     FOR rec IN (SELECT DISTINCT DEPT FROM EMPLOYES ORDER BY DEPT)
4     LOOP
5         DBMS_OUTPUT.PUT_LINE(
6             'Dept ' || rec.DEPT || ' : '
7             || nb_employees_dept(rec.DEPT) || ' employe(s)'
8         );
9     END LOOP;
10 END;
11 /

```

8.10 Exercice 10 – Suppression des petits salaires

```

1 CREATE OR REPLACE PROCEDURE suppr_petits_salaires (
2     p_seuil IN NUMBER
3 ) IS
4     TYPE t_noms IS TABLE OF EMPLOYES.ENAME%TYPE;
5     v_noms t_noms;
6 BEGIN
7     -- Recuperer les noms avant suppression
8     SELECT ENAME BULK COLLECT INTO v_noms
9     FROM EMPLOYES WHERE SALARY < p_seuil;
10
11     DELETE FROM EMPLOYES WHERE SALARY < p_seuil;
12
13     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employe(s) supprime(s)
14                          :');

```



```

14     FOR i IN 1..v_noms.COUNT LOOP
15         DBMS_OUTPUT.PUT_LINE(' - ' || v_noms(i));
16     END LOOP;
17     COMMIT;
18 EXCEPTION
19     WHEN OTHERS THEN
20         ROLLBACK;
21         DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);
22 END;
23 /

```

Questions supplémentaires

1. Pourquoi est-il dangereux d'utiliser COMMIT dans une procédure appelée par un programme externe ?

Si la procédure fait partie d'une transaction plus large gérée par l'application externe, un COMMIT interne valide des modifications partielles qui n'auraient pas dû l'être. En cas d'erreur après le COMMIT, il est impossible d'annuler les suppressions déjà validées.

2. Clause RETURNING pour lister les noms supprimés

Déjà implémenté ci-dessus avec BULK COLLECT avant le DELETE.

3. Version avec SAVEPOINT

```

1 CREATE OR REPLACE PROCEDURE suppr_petits_salaires_sp (
2     p_seuil IN NUMBER
3 ) IS
4 BEGIN
5     SAVEPOINT avant_suppression;
6
7     DELETE FROM EMPLOYES WHERE SALARY < p_seuil;
8     DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employe(s) supprime(s).');
9     COMMIT;
10 EXCEPTION
11     WHEN OTHERS THEN
12         ROLLBACK TO avant_suppression;
13         DBMS_OUTPUT.PUT_LINE('Erreur, annulation : ' || SQLERRM);
14 END;
15 /

```

8.11 Tableau de synthèse

Critère	Procédure	Fonction
Retourne une valeur	Non (sauf OUT)	Oui (via RETURN)
Appel dans SQL	Non	Oui
Transaction possible	Oui	Déconseillé
Utilisation typique	Actions, mises à jour	Calculs, transformations

TP 9 – TP — Les Triggers PL/SQL sous Oracle

Énoncé

Schéma utilisé : tables etudiant, module, prerequis, inscription, examen, passer.

9.1 Exercices supplémentaires

Contrainte 10 – Interdire l'inscription si module déjà validé

```
1 CREATE OR REPLACE TRIGGER trg_inscription_module_valide
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     v_note_max NUMBER;
6     v_note_min NUMBER;
7 BEGIN
8     SELECT note_min INTO v_note_min
9     FROM module WHERE code_module = :NEW.code_module;
10
11     SELECT MAX(pa.note) INTO v_note_max
12     FROM passer pa
13     JOIN examen e ON pa.id_examen = e.id_examen
14     WHERE pa.id_etudiant = :NEW.id_etudiant
15           AND e.code_module = :NEW.code_module;
16
17     IF v_note_max IS NOT NULL AND v_note_max >= v_note_min THEN
18         RAISE_APPLICATION_ERROR(-20010,
19             'L''etudiant a deja valide ce module.');
```

```
20     END IF;
21 EXCEPTION
22     WHEN NO_DATA_FOUND THEN NULL;
23 END;
24 /
```

Contrainte 11 – Pas d'examen dans une fenêtre de 7 jours

```
1 CREATE OR REPLACE TRIGGER trg_examen_conflit_planning
2 BEFORE INSERT ON examen
3 FOR EACH ROW
4 DECLARE
5     v_count NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_count
8     FROM examen
9     WHERE code_module = :NEW.code_module
10           AND ABS(:NEW.date_examen - date_examen) <= 7;
11
12     IF v_count > 0 THEN
13         RAISE_APPLICATION_ERROR(-20011,
14             'Un examen existe deja pour ce module '
15             || 'dans les 7 jours autour de cette date.');
```

```

16     END IF;
17 END;
18 /

```

Contrainte 12 – Moyenne calculée uniquement si tous les examens sont passés

```

1  -- Table de journalisation des avertissements
2  CREATE TABLE journal_erreurs (
3      id_etudiant  NUMBER,
4      message      VARCHAR2(200),
5      date_log     DATE DEFAULT SYSDATE
6  );
7
8  CREATE OR REPLACE TRIGGER trg_moyenne_complete
9  AFTER INSERT OR UPDATE OR DELETE ON passer
10 FOR EACH ROW
11 DECLARE
12     v_id_etudiant  NUMBER;
13     v_nb_inscrits  NUMBER;
14     v_nb_passes    NUMBER;
15     v_moyenne      NUMBER;
16     PRAGMA AUTONOMOUS_TRANSACTION;
17 BEGIN
18     IF INSERTING OR UPDATING THEN
19         v_id_etudiant := :NEW.id_etudiant;
20     ELSE
21         v_id_etudiant := :OLD.id_etudiant;
22     END IF;
23
24     -- Nombre de modules auxquels l'etudiant est inscrit
25     SELECT COUNT(*) INTO v_nb_inscrits
26     FROM inscription
27     WHERE id_etudiant = v_id_etudiant;
28
29     -- Nombre de modules pour lesquels il a passe au moins un examen
30     SELECT COUNT(DISTINCT e.code_module) INTO v_nb_passes
31     FROM passer pa
32     JOIN examen e ON pa.id_examen = e.id_examen
33     WHERE pa.id_etudiant = v_id_etudiant;
34
35     IF v_nb_passes < v_nb_inscrits THEN
36         -- Pas tous les examens passes : moyenne reste NULL
37         UPDATE etudiant SET moyenne = NULL
38         WHERE id_etudiant = v_id_etudiant;
39
40         INSERT INTO journal_erreurs (id_etudiant, message)
41         VALUES (v_id_etudiant,
42             'Moyenne non calculee : examens manquants ('
43             || v_nb_passes || '/' || v_nb_inscrits || ')');

```

```

44         COMMIT;
45     ELSE
46         -- Tous les examens passes : calcul de la moyenne ponderee
47         SELECT SUM(pa.note * e.coefficient) / SUM(e.coefficient)
48         INTO v_moyenne
49         FROM passer pa
50         JOIN examen e ON pa.id_examen = e.id_examen
51         WHERE pa.id_etudiant = v_id_etudiant
52               AND pa.note IS NOT NULL;
53
54         UPDATE etudiant SET moyenne = v_moyenne
55         WHERE id_etudiant = v_id_etudiant;
56         COMMIT;
57     END IF;
58 END;
59 /

```

Contrainte 13 – Interdire la suppression d’un module référencé comme prérequis

```

1 CREATE OR REPLACE TRIGGER trg_module_no_delete_if_prereq
2 BEFORE DELETE ON module
3 FOR EACH ROW
4 DECLARE
5     v_count NUMBER;
6 BEGIN
7     SELECT COUNT(*) INTO v_count
8     FROM prerequis
9     WHERE prerequis = :OLD.code_module;
10
11     IF v_count > 0 THEN
12         RAISE_APPLICATION_ERROR(-20013,
13             'Le module ' || :OLD.code_module
14             || ' est reference comme prerequis '
15             || 'et ne peut pas etre supprime. ');
16     END IF;
17 END;
18 /

```

Conclusion générale

Ces neuf travaux pratiques ont couvert l’essentiel de la programmation PL/SQL sous Oracle : des sous-requêtes et vues SQL jusqu’aux triggers, en passant par les curseurs, les procédures et les fonctions. Chaque TP a renforcé une compétence précise tout en s’appuyant sur les acquis précédents, formant ainsi une progression cohérente de la manipulation de données vers la logique métier embarquée côté serveur.